API Performance & Security Audit Report

PawGuard Backend — FastAPI v1 API Surface

PawGuard Platform | pawguard-backend (FastAPI + PostgreSQL + Redis + ARQ) | Prepared for Team Review

---

1. Executive Summary

The backend follows a layered architecture (Router → Service → Repository → DB), enforces RBAC/PBAC, uses RS256 JWT access tokens, Argon2id password hashing, Redis-backed rate limiting, structured logging, and ARQ background workers. Overall posture: strong for an early-stage product, with a handful of production hardening items before scaling.

2. Security Controls In Place

Area	Implementation	Status
Authentication	RS256 JWT, 15-min access tokens, opaque SHA-256 refresh tokens	PASS
Password storage	Argon2id (t=2, m=19456, p=1)	PASS
MFA	TOTP secrets encrypted at rest with Fernet	PASS
Authorization	RBAC + PBAC, Redis-cached permissions, explicit admin bypass	PASS
Rate limiting	Redis sliding-window per user/IP (auth + broadcast endpoints)	PASS
Audit logging	AuthAuditEventType events via AuditService	PASS
Security headers	CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Permissions-Policy	PASS
Body size limits	RequestBodySizeMiddleware rejects > 10 MB	PASS
PII masking	core.pii utilities for email/phone/name/IP	PASS
QR safety tags	Raw token returned once; only SHA-256 hash stored	PASS

3. Security — Recommended Before Scale

Priority	Item	Action
HIGH	CORS wildcard in local defaults	allowed_hosts: "*" is unsafe for production — override to exact domains
HIGH	JWT key rotation	Store PEM in env/KMS; rotate on a schedule, keep prior public key during the window
MED	Rate-limit gaps	Add rate limits to expensive unprotected reads (/rescue/dispatches, /shelter/transfers, /lost, /donations)
MED	Input sanitization	HTML/JS-escape free-text fields echoed back in responses
MED	Dependency scanning	Pin + scan requirements.txt (pip-audit / Snyk)
LOW	Refresh-token binding	Bind to device fingerprint to reduce replay risk

4. Performance — Strengths

- Async SQLAlchemy + asyncpg, non-blocking request handling throughout.
- Redis caching for RBAC permission lookups (5-min TTL).
- ARQ background workers for email, reminders, and lost-pet broadcasts — keeps the API responsive.
- Companion-pet reminders and lost-pet broadcasts fan out in 500-user batches to avoid memory spikes.
- Per-request metrics counters and latency histograms already emitted.

5. Performance — Recommended Before Scale

Priority	Item	Action
HIGH	N+1 queries	Audit list endpoints with nested relations; add selectinload/joinedload where missing
HIGH	DB latency	Remote integration tests run ~16s/op — add pooling (PgBouncer) or move DB closer to app
MED	Hot-read caching	Cache reference data (roles, vet clinics) with TTL
MED	Cursor pagination	Replace offset pagination on high-volume feeds (lost/found, notifications, audit logs)
LOW	Indexes	Composite indexes on status+created_at, user_id+status, microchip_id;

		geospatial index if PostGIS is adopted

6. Pet-Vertical Specific Notes

- POST /lost-found/lost/{id}/broadcast queues an ARQ job and sets broadcasted_at exactly once — no duplicate alert waves.
- Appointment creation uses a DB-level exclusion constraint to prevent double-booking a clinic slot.
- Safety-tag scan endpoint: rate-limited (20/60s per IP) and returns only PII-free emergency data.

Source: docs/API_SECURITY_PERFORMANCE_AUDIT.md (dated 2026-08-09), reviewed and condensed for this session.